PRACTICAL NO.1(A)

Aim:- Introduction to Business Aanalysis(EDA and customer Data Analysis).

Code:-

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df=pd.read_csv("Practical_1.csv")
df.head()
```

Press **Shift** + **Enter** to execute the code.

Output:-

	Invoice_ID	Branch	City	Customer Type	Gender	Product Line	Unit Price	Quantity	Total	Date
0	INV001	A	Mumbai	Member	Female	Food & Beverages	650	2	1300	02-02-2026
1	INV002	B	Mumbai	Normal	Male	Electronics	486	4	1944	26-03-2026
2	INV003	C	Pune	Member	Female	Fashion	1475	10	14750	02-04-2026
3	INV004	A	Pune	Normal	Male	Health & Beauty	574	5	2870	19-05-2026
4	INV005	B	Mumbai	Member	Female	Food & Beverages	567	3	1701	02-06-2026

df.info()
(Gives all the information like null value count and data type of each column).

Output:-

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 10 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Invoice_ID     5 non-null      object
 1   Branch         5 non-null      object
 2   City           5 non-null      object
 3   Customer Type  5 non-null      object
 4   Gender         5 non-null      object
 5   Product Line   5 non-null      object
 6   Unit Price     5 non-null      int64
 7   Quantity       5 non-null      int64
 8   Total          5 non-null      int64
 9   Date           5 non-null      object
dtypes: int64(3), object(7)
memory usage: 532.0+ bytes
```

df.isnull().sum()
(Give the total number of null values present in each column).

Output:-

```
Invoice_ID       0
Branch           0
City             0
Customer Type    0
Gender           0
Product Line     0
Unit Price       0
Quantity         0
Total            0
Date             0
dtype: int64
```

df.describe()
(Gives all the descriptive statistics like mean, std dev, min, max, 25%, 50%, 75%).

Output:-

	Unit Price	Quantity	Total
count	5.000000	5.000000	5.000000
mean	750.400000	4.800000	4513.000000
std	409.202028	3.114482	5751.653936
min	486.000000	2.000000	1300.000000
25%	567.000000	3.000000	1701.000000
50%	574.000000	4.000000	1944.000000
75%	650.000000	5.000000	2870.000000
max	1475.000000	10.000000	14750.000000

df["Customer Type"].value_counts()
(Counts the total number customer type in dataset).

Output:-

```
Customer Type
Member    3
Normal    2
Name: count, dtype: int64
```

df["Gender"].value_counts()
(Counts the total number of customers in each type of gender).

Output:-

```
Gender
Female    3
Male      2
Name: count, dtype: int64
```

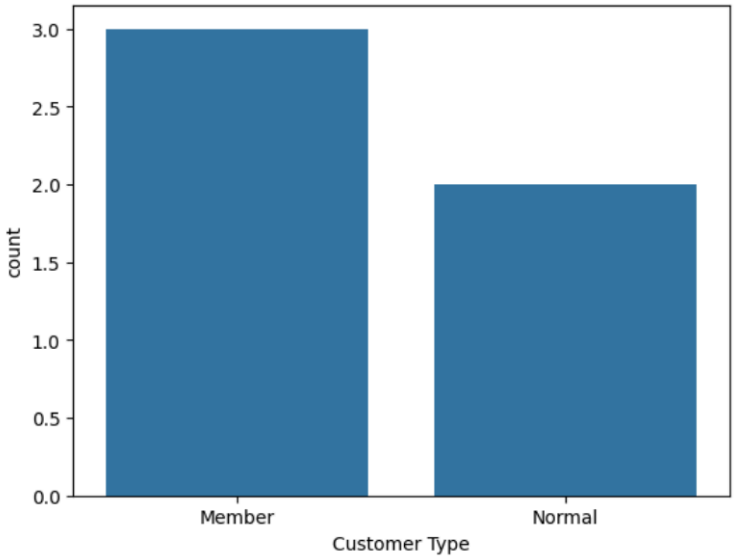
df["Product Line"].value_counts()
(It counts total number of product in each product line).

Output:-

```
Product Line
Food & Beverages    2
Electronics         1
Fashion             1
Health & Beauty     1
Name: count, dtype: int64
```

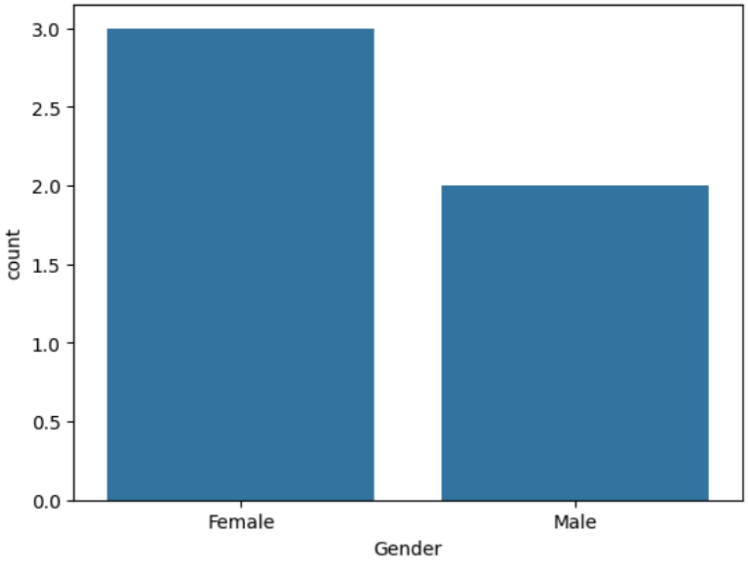
```
sns.countplot(x="Customer Type", data=df)
plt.show()
```

Output:-



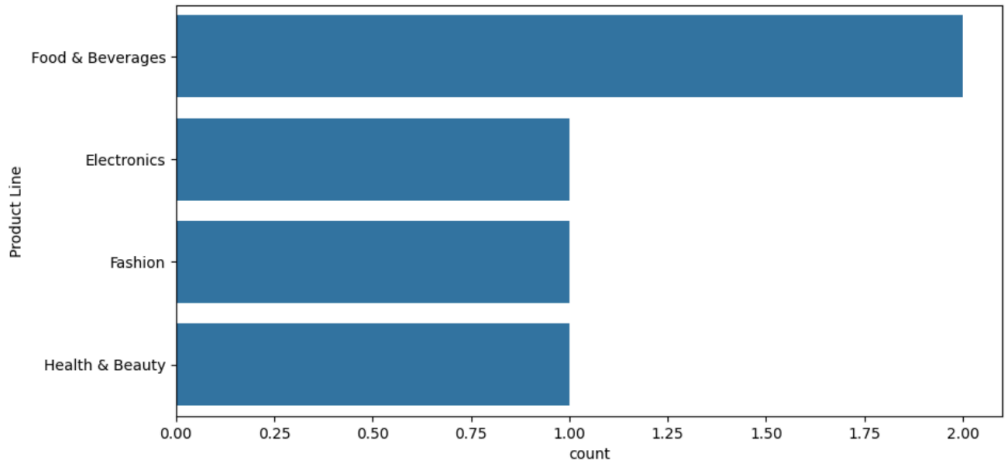

```
sns.countplot(x="Gender",data=df)
plt.show()
```

Output:-



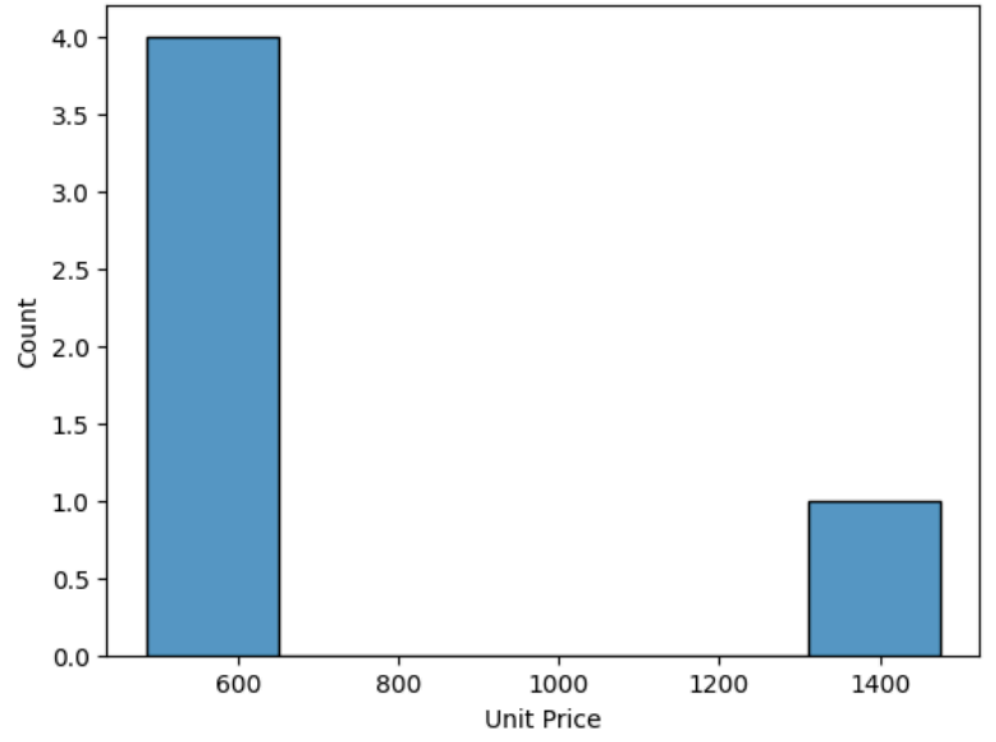
```
plt.figure(figsize=(10,5))
sns.countplot(y="Product Line",data=df)
plt.show()
```

Output:-



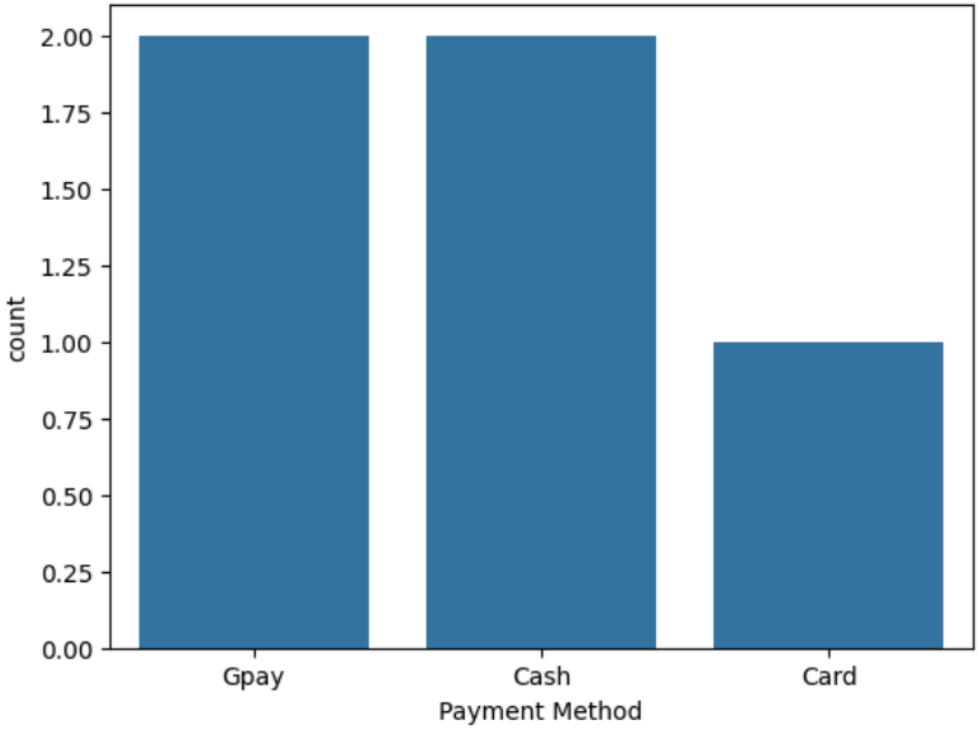

```
sns.histplot(df["Unit Price"],bins=6)
plt.show()
```

Output:-



```
sns.countplot(x="Payment Method",data=df)
plt.show()
```

Output:-



Conclusion:-

This practical introduced the basics of **Business Analysis using Data Analysis (EDA)**. The sales dataset was successfully loaded and analyzed using Python libraries such as Pandas, Matplotlib, and Seaborn. The dataset was checked for its structure, data types, and missing values, and no missing values were found. Statistical measures such as mean, minimum, maximum, and standard deviation were obtained using descriptive analysis. Customer type, gender, product line, unit price, and payment-related data were analyzed using different visualizations. Count plots and histograms helped in understanding customer and product distributions. Overall, this practical demonstrated how data analysis and visualization can provide useful insights for better understanding business and customer data.

PRACTICAL NO.1(B)

Q.1) Who are the most valuable customers?
→ The customer with INV008 is the most valuable customer as he purchased 6 quantity of Electronic products each worth Rs.1500/- bringing his total amount to Rs.9000/-. He purchased all the product on 18-11-2026 and done payment using card and is normal customer living in Mumbai.

Q.2)Which product category has the highest sales?
→ The product with highest sales is Food & Beverage with total sales of 16 quantity each of worth Rs.600/- making total sales amount to Rs.9420/-.

Q.3)Which region generates the highest revenue?
→ The Mumbai region produces the highest revenue by generating total amount of Rs.17370\-.

Q.4)Which Payment method is used most often?
→ GooglePay(GPay) payment method is used most ften being used by 4 times out of 10 records.

Q.5)Are there any buying patterns?
→ Yes, there are three buying patterns GPay, Cash and Card, with most preffered option as GPay.

PRACTICAL NO 2. Describing the Distribution of a variable.

A)Aim:- To calculate descriptive statistics for a business dataset.

Dataset:- Sales.CSV

	Product	Category	Sales
1	Laptop	Electronics	78000
2	Mobile	Electronics	40000
3	Chair	Furniture	10000
4	Table	Furniture	15000
5	Printer	Electronics	35000
6	Sofa	Furniture	45000
7	AC	Electronics	96000
8	Fan	Electronics	25000
9	Cupboard	Furniture	30000
10	TV	Electronics	88000
11	Bed	Furniture	56000
12	Washing Machine	Electronics	76000
13	Refrigerator	Electronics	87000
14	Desk	Electronics	20000
15	Bookshelf	Furniture	5000

Code:-

#Import Libraries

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

#Load dataset

```
df=pd.read_csv("sales2.csv")
```

#Printing dataset

```
print("dataset")
print(df)
```

```
Dataset
            Product     Category  Sales
0            Laptop  Electronics  78000
1            Mobile  Electronics  40000
2             Chair    Furniture  10000
3             Table    Furniture  15000
4           Printer  Electronics  35000
5              Sofa    Furniture  45000
6                AC  Electronics  96000
7               Fan  Electronics  25000
8         Cupboard    Furniture  30000
9                TV  Electronics  88000
10              Bed    Furniture  56000
11  Washing Machine  Electronics  76000
12     Refrigerator  Electronics  87000
13             Desk  Electronics  20000
14        Bookshelf    Furniture   5000
```

Descriptive Statistics

```
print("\nDescroptive Statistics")
print(df["Sales"].describe())
```

```
Descriptive Statistics
count       15.000000
mean     47066.666667
std      30961.881326
min       5000.000000
25%      22500.000000
50%      40000.000000
75%      77000.000000
max      96000.000000
Name: Sales, dtype: float64
```

Mean, Median, Mode

```
print("\nMean:",df["Sales"].mean())
print("\nMedian:",df["Sales"].median())
print("\nMode:",df["Sales"].mode())
```

```
Mean: 47066.666666666664

Median: 40000.0

Mode: 0      5000
1     10000
2     15000
3     20000
4     25000
5     30000
6     35000
7     40000
8     45000
9     56000
10    76000
11    78000
12    87000
13    88000
14    96000
Name: Sales, dtype: int64
```

#Variance, Standard Deviation

```
print("\nVariance:",df["Sales"].var())
print("\nStandard Deviation:",df["Sales"].std())
```

```
Variance: 958638095.2380952

Standard Deviation: 30961.88132588353
```

#Minimum, Maximum

```
print("\nMinimum:",df["Sales"].min())
print("\nMaximum:",df["Sales"].max())
```

```
Minimum: 5000

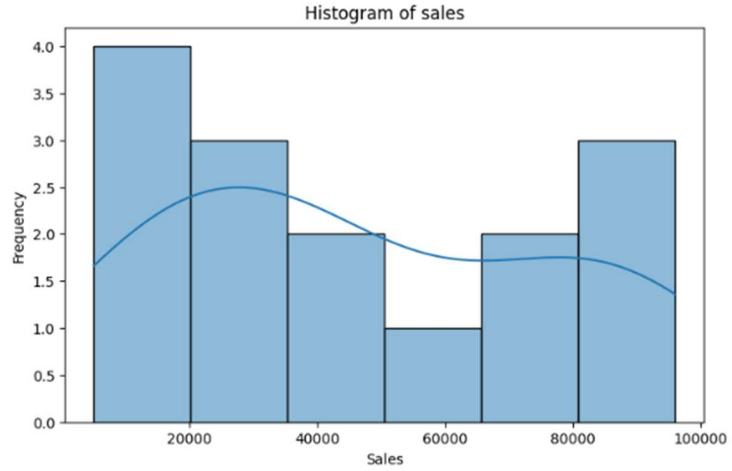
Maximum: 96000
```

B)Aim:- Visualize the distribution of variable using histogram and box plot

Code:- (Histogram)

```
plt.figure(figsize=(8,5))
sns.histplot(df["Sales"],bins=6,kde=True)
plt.title("Histogram of sales")
plt.xlabel("Sales")
plt.ylabel("Frequency")
```

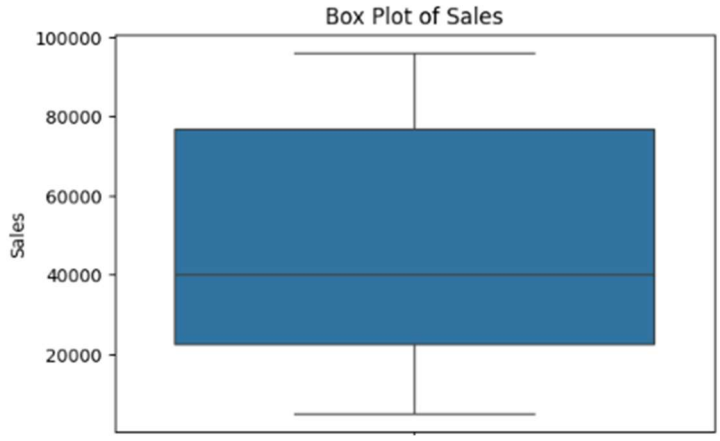
Output:-



Code:- (Box Plot)

```
plt.figure(figsize=(6,4))
sns.boxplot(y=df["Sales"])
plt.title("Box Plot of Sales")
plt.show()
```

Output:-



Conclusion:-

The sales dataset was successfully analyzed using various descriptive statistical methods. Mean, median, mode, variance, standard deviation, minimum, and maximum values were calculated to understand the sales data in detail. The distribution of sales was also represented visually using a histogram and a box plot, which helped in understanding the frequency, spread, and variation of the data. Overall, this practical provided a clear understanding of how descriptive statistics and graphical methods can be used to analyze and interpret a business dataset.

PRACTICAL 7. Sampling and Sampling Distributions

A.) Aim:- To conduct a survey and collect data from a sample population using proper sampling techniques.

B.) Aim:- To use the Central Limit Theorem(CLT) to analyze the sampling distribution of a sample mean and estimate population parameters.

Code:-

```
import numpy as np
import matplotlib.pyplot as plt

#Population data
population=np.array([
    2,3,4,2,5,3,4,6,3,5,
    2,4,3,5,4,6,3,4,5,2,
    3,4,5,3,6,4,2,5,4,3,
    5,6,3,4,2,5,3,4,6,5,
    2,3,4,5,3,6,4,2,5,4
])

#Population_Mean
population_mean=np.mean(population)
print("Population Mean:",population_mean)

#Number of Samples
num_samples=1000

#Sample Size
sample_size=10

#Store sample means
sample_means=[]

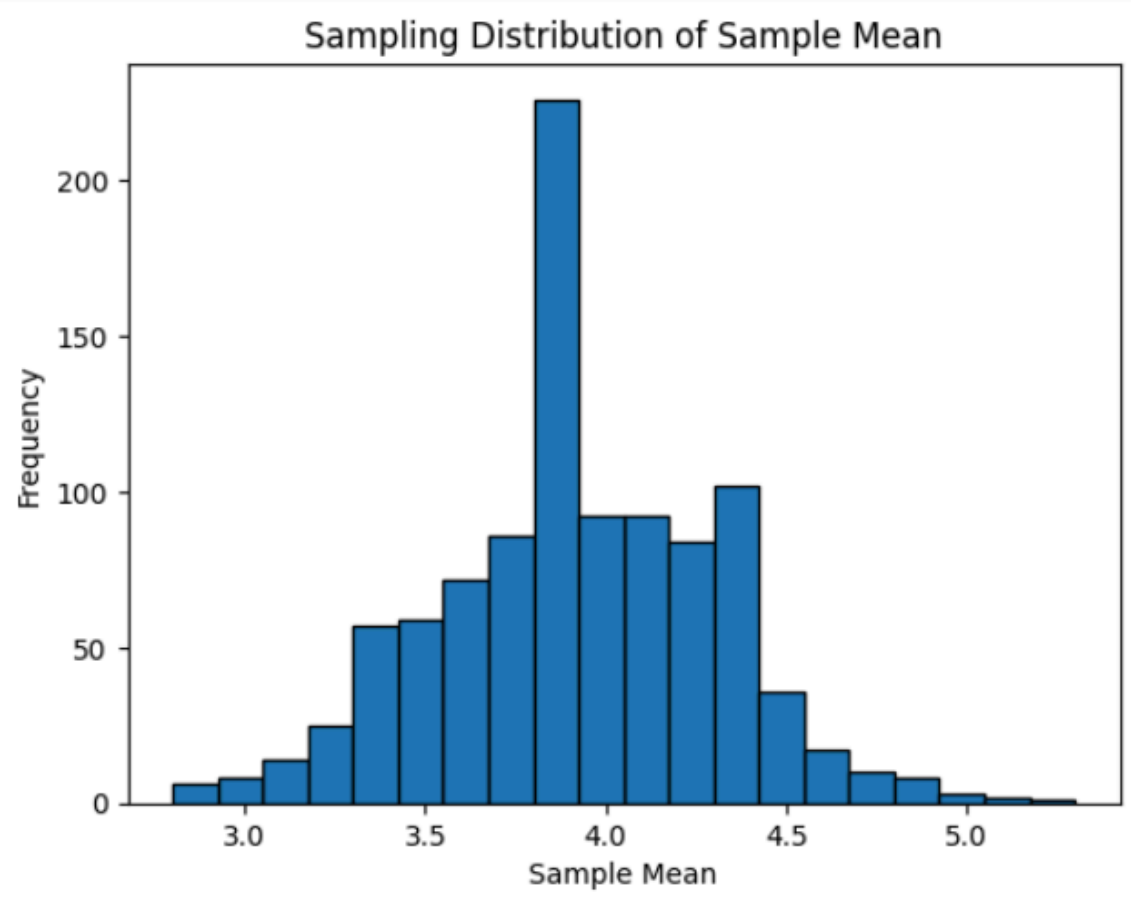
#Generate samples
for i in range(num_samples):
    sample=np.random.choice(population,
                            size=sample_size)
    sample_mean=np.mean(sample)
    sample_means.append(sample_mean)
```

```
#Mean of sampling distribution
sampling_mean=np.mean(sample_means)
print("Mean of Sampling Distribution:",sampling_mean)

#Plot sampling distribution
plt.hist(sample_means,bins=20,edgecolor="black")
plt.xlabel("Sample Mean")
plt.ylabel("Frequency")
plt.title("Sampling Distribution of Sample Mean")
plt.show()
```

Output:-

```
Population Mean: 3.9
Mean of Sampling Distribution: 3.9083999999999994
```



PRACTICAL NO.3

Aim:- Findng Relationship among variables.

Objectives:-

1. Perform **correlation analysis** to determine the strength and direction of relationships between pairs of variables.
2. Perform **regression analysis** to identify the relationship between an independent variable and a dependent variable.

Code:-

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns

from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score,mean_squared_error

print("="*70)
print("DATASET")
print("="*70)

print(df.to_string(index=False))
```

```
======================================================================
DATASET
======================================================================
Employee  Training_Hours  Experience_Years  Attendance_Percentage  Productivity_Score
      E1               5                 1                     82                  52
      E2               8                 2                     85                  56
      E3              12                 3                     88                  61
      E4              15                 4                     90                  65
      E5              18                 5                     87                  68
      E6              20                 6                     92                  72
      E7              23                 7                     94                  75
      E8              25                 8                     91                  78
      E9              28                 9                     96                  82
     E10              30                10                     95                  85
```

```
print("\n"+"="*70)
print("DATASET INFORMATION")
print("="*70)

print("Number of rows:",df.shape[0])
```

```
print("Number of column:",df.shape[1])

print("\nMissing Values:")
print(df.isnull().sum())
```

```
======================================================================
DATASET INFORMATION
======================================================================
Number of rows: 10
Number of column: 5

Missing Values:
Employee                0
Training_Hours          0
Experience_Years        0
Attendance_Percentage   0
Productivity_Score      0
dtype: int64
```

```
print("\n"+"="*70)
print("STATISTICAL SUMMARY")
print("="*70)

print(df.describe())
```

```
======================================================================
STATISTICAL SUMMARY
======================================================================
       Training_Hours  Experience_Years  Attendance_Percentage  \
count       10.000000          10.00000              10.000000
mean        18.400000           5.50000              90.000000
std          8.395766           3.02765               4.521553
min          5.000000           1.00000              82.000000
25%         12.750000           3.25000              87.250000
50%         19.000000           5.50000              90.500000
75%         24.500000           7.75000              93.500000
max         30.000000          10.00000              96.000000

       Productivity_Score
count           10.000000
mean            69.400000
std             10.996969
min             52.000000
25%             62.000000
50%             70.000000
75%             77.250000
max             85.000000
```

```
#CORRELATION ANALYSIS
numerical_data=df[
['Training_Hours',
 'Experience_Years',
 'Attendance_Percentage',
 'Productivity_Score']
]

correlation_matrix=numerical_data.corr()

print("\n"+"="*70)
print("CORRELATION MATRIX")
print("="*70)

print(correlation_matrix.round(3))
```

```
======================================================================
CORRELATION MATRIX
======================================================================
                     Training_Hours  Experience_Years  \
Training_Hours                1.000             0.997
Experience_Years              0.997             1.000
Attendance_Percentage         0.928             0.917
Productivity_Score            0.999             0.998

                     Attendance_Percentage  Productivity_Score
Training_Hours                       0.928               0.999
Experience_Years                     0.917               0.998
Attendance_Percentage                1.000               0.932
Productivity_Score                   0.932               1.000
```

```
correlation=df['Training_Hours'].corr(
   df['Productivity_Score']
)

print("\n"+"="*70)
print("TRAINING HOURS VS PRODUCTIVITY")
print("="*70)

print("X=Training Hours")
print("Y=Productivity Score")
print("Pearson Correlation Coefficient:",round(correlation,4))
```

```
======================================================================
TRAINING HOURS VS PRODUCTIVITY
======================================================================
X=Training Hours
Y=Productivity Score
Pearson Correlation Coefficient: 0.9993
```

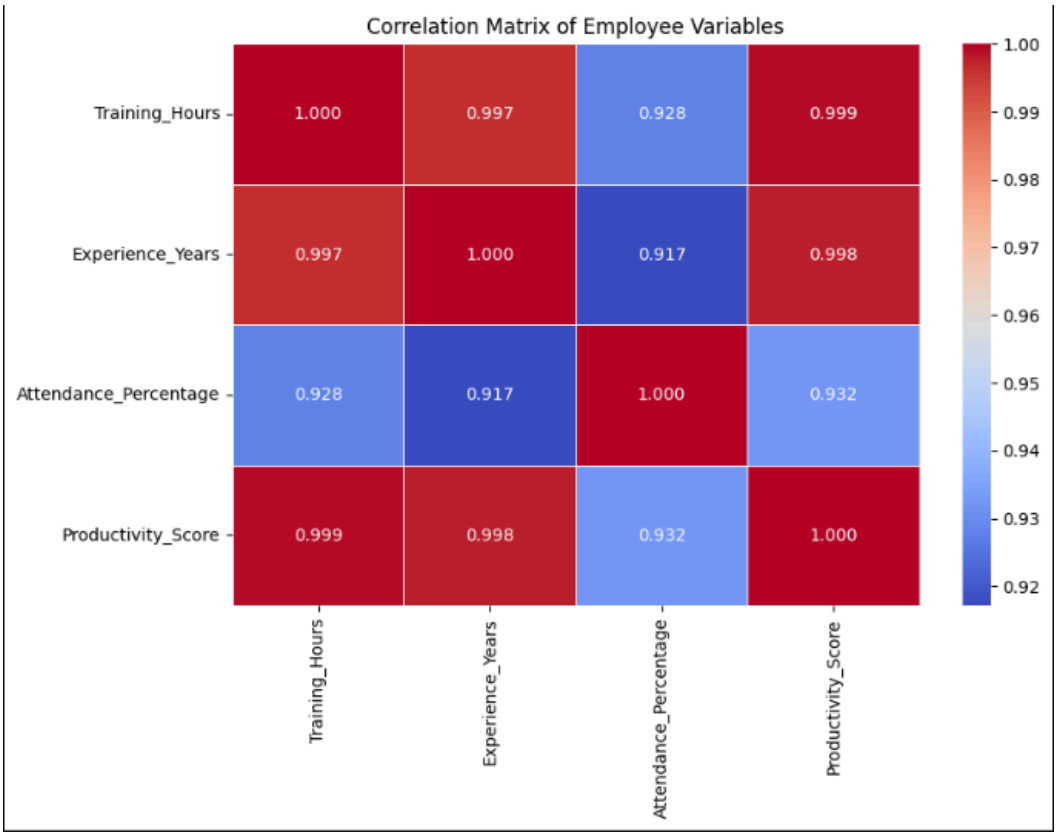
```
if correlation>0:
    print("Interpretation: Strong positivite relationship.")
elif correlation<0:
    print("Interpretation: Negative relationship.")
else:
    print("Interpretation: No relationship.")
```

```
Interpretation: Strong positivite relationship.
```

```
plt.figure(figsize=(9,6))

sns.heatmap(
    correlation_matrix,
    annot=True,
    cmap='coolwarm',
    fmt='.3f',
    linewidths=0.5
)

plt.title("Correlation Matrix of Employee Variables")
plt.tight_layout()
plt.show()
```




```
X=df[['Training_Hours']]
Y=df[['Productivity_Score']]

model=LinearRegression()
model.fit(X,Y)

slope=model.coef_[0]
intercept=model.intercept_

print("\n"+"="*70)
print("LINEAR REGRESSION RESULTS")
print("="*70)

print("Independent Variable: Training Hours")
print("Dependent Variable: Productivity Score")

print("\nSlope/Coefficient:",round(slope,4))
print("Intercept:",round(intercept,4))

print("\nRegression Equation:")
```

```
print(
    f"Productivity Score={intercept:.2f}"
    f"+{slope:.2f} x Training Hours"
)
y_pred=model.predict(X)
```

```
======================================================================
LINEAR REGRESSION RESULTS
======================================================================
Independent Variable: Training Hours
Dependent Variable: Productivity Score

Slope/Coefficient: 2.35
Intercept: 40.5

Regression Equation:
Productivity Score=40.50+2.35 x Training Hours
```

```
df['Predicted_Productivity']=y_pred

print("\n"+"="*70)
print("ACTUAL VS PREDICTED PRODUCTIVITY")
print("="*70)

print(
    df[
      ['Employee',
       'Training_Hours',
       'Productivity_Score',
       'Predicted_Productivity']
      ].round(2).to_string(index=False))
```

```
======================================================================
ACTUAL VS PREDICTED PRODUCTIVITY
======================================================================
Employee  Training_Hours  Productivity_Score  Predicted_Productivity
      E1              5                  52                   51.86
      E2              8                  56                   55.79
      E3             12                  61                   61.02
      E4             15                  65                   64.95
      E5             18                  68                   68.88
      E6             20                  72                   71.49
      E7             23                  75                   75.42
      E8             25                  78                   78.04
      E9             28                  82                   81.97
     E10             30                  85                   84.58
```

```
new_training_hours=pd.DataFrame({
   'Training_Hours':[22]
   })
prediction=model.predict(new_training_hours)
print("\n"+"="*70)
print("PREDICTION")
print("="*70)

print(
   "Trainng Hours =22"
)
```

```
======================================================================
PREDICTION
======================================================================
Trainng Hours =22
```

```
r2=r2_score(Y,y_pred)

print("\n"+"="*70)
print("MODEL PERFORMANCE")
print("="*70)

print("R-squared:",round(r2,4))
print("R-squared Percentage:",round(r2*100,2),"%")
```

```
==========================================================================
MODEL PERFORMANCE
==========================================================================
R-squared: 0.9987
R-squared Percentage: 99.87 %
```

```
mse=mean_squared_error(Y,y_pred)
print("Mean Squared Error:",round(mse,4))
```

```
Mean Squared Error: 0.1445
```

```
plt.figure(figsize=(9,6))

plt.scatter(
    df['Training_Hours'],
    df['Productivity_Score'],
    color='blue',
    s=80
)

plt.xlabel('Training Hours')
plt.ylabel('Productivity Score')
plt.title('Training Hours VS Productivity Score')

plt.grid(True)
plt.tight_layout()
plt.show()
```



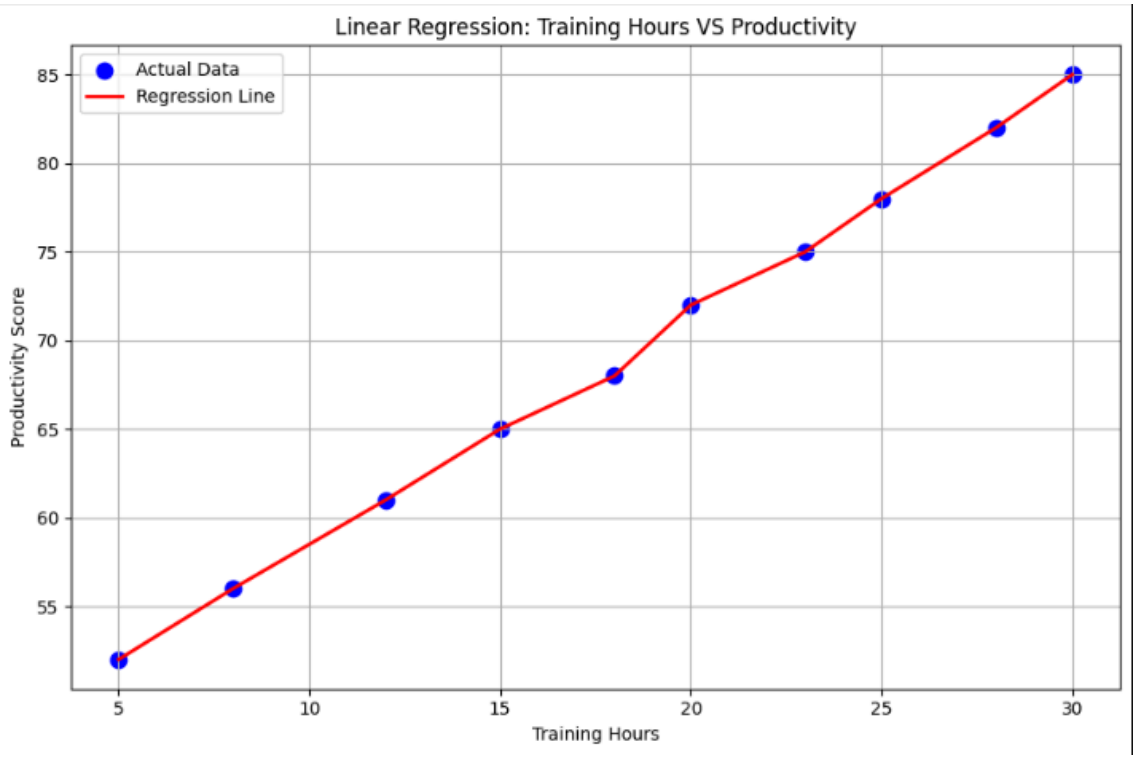

```
plt.figure(figsize=(9,6))

plt.scatter(
    X,
    Y,
    color='blue',
    s=80,
    label='Actual Data'
)

plt.plot(
    X,
    Y,
    color='red',
    linewidth=2,
    label='Regression Line'
)

plt.xlabel('Training Hours')
plt.ylabel('Productivity Score')
plt.title('Linear Regression: Training Hours VS Productivity')
```

```
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```




```
print("\n"+"="*70)
print("FINAL CONCLUSION")
print("="*70)

print("""
1.Correlation analysis was performed using Training Hours,Attendance, and Productivity Score.

2.The correlation between Training Hours and Productivity Score is approximately 0.999, indicating a very strong positive relationship.

3.Linear regression was performed with Training Hours as the independent variable and Productivity Score as the dependent variable.

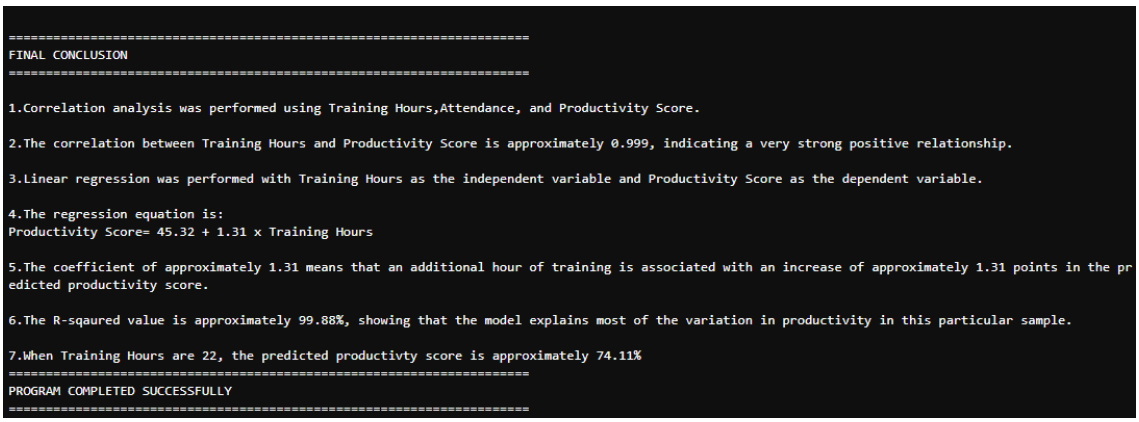
4.The regression equation is:
Productivity Score= 45.32 + 1.31 x Training Hours
```

5.The coefficient of approximately 1.31 means that an additional hour of training is associated with an increase of approximately 1.31 points in the predicted productivity score.

6.The R-sqaured value is approximately 99.88%, showing that the model explains most of the variation in productivity in this particular sample.

7.When Training Hours are 22, the predicted productivty score is approximately 74.11%""")

```
print("="*70)
print("PROGRAM COMPLETED SUCCESSFULLY")
print("="*70)
```



Conslusion:-

The practical successfully demonstrated how correlation and regression analysis can be used to find relationships among variables. Correlation analysis helped determine the strength and direction of the relationship between pairs of variables. Regression analysis helped identify how an independent variable influences or predicts a dependent variable. These techniques are useful for understanding patterns, making predictions, and supporting data-driven decision-making. Thus, the practical provided a clear understanding of the relationship among variables using statistical analysis.

PRACTICAL NO 4. BUSINESS INTELLIGENCE TOOLS FOR DATA ANALYSIS

Aim:- To utilize Tableau as a Business Intelligence tool to extract insights from a dataset and create interactive visualizations for effective data analysis.

Main Purpose:- The main purpose of this practical is to learn how **Tableau converts raw data into meaningful charts, graphs, and dashboards** so that business information can be understood easily and decisions can be made from it.

Dataset:-

You can use a simple **Retail Store Dataset**.

Order ID	Date	Product	Category	Region	Sales	Quantity	Profit
1001	01-01-2025	Laptop	Electronics	Mumbai	55000	2	8000
1002	03-01-2025	Mobile	Electronics	Pune	30000	3	4500
1003	05-01-2025	Chair	Furniture	Mumbai	12000	4	2500
1004	08-01-2025	Table	Furniture	Nashik	18000	2	3500
1005	10-01-2025	Printer	Electronics	Pune	15000	2	2200
1006	12-01-2025	Desk	Furniture	Mumbai	22000	3	4000
1007	15-01-2025	Keyboard	Accessories	Nashik	5000	5	1200
1008	18-01-2025	Mouse	Accessories	Pune	3000	6	800
1009	20-01-2025	Monitor	Electronics	Mumbai	25000	2	3500
1010	23-01-2025	Sofa	Furniture	Pune	35000	1	6000

Step-by-Step Procedure in Tableau

Step 1: Open Tableau

1. Open **Tableau Public** or **Tableau Desktop**.
2. Click **Connect to Data**.
3. Select **Microsoft Excel** if your dataset is in Excel.
4. Select your Excel file.
5. Select the required sheet.
6. Click **Sheet 1** at the bottom.

Step 2: Understand the Data

After opening the worksheet, you will see fields such as:

Dimensions

- Order ID
- Date
- Product
- Category
- Region

Measures

- Sales
- Quantity
- Profit

Simple meaning:-

Dimension = information used to categorize data.

Example:

Product, Category, Region

Measure = numerical information that can be calculated.

Example:

Sales, Quantity, Profit

Step 3: Create a Sales by Category Chart

We want to answer:

> **Which category has the highest sales?**

1. Open **Sheet 1**.
2. Drag **Category → Columns**.
3. Drag **Sales → Rows**.
4. Tableau automatically creates a **bar chart**.
5. Click **Show Me** if necessary and select **Bar Chart**.
6. Click **Sort Descending** to arrange categories from highest to lowest sales.

You will get a chart showing:

Category → Sales

Sales by Category

Illustrative example showing how category sales can be compared in a BI visualization.

035K70K105K140K
Electronics
Furniture
Accessories

Step 4: Create Sales by Region

Now we want to know:

> **Which region generates the highest sales?**

1. Create a new worksheet.
2. Drag **Region → Columns**.
3. Drag **Sales → Rows**.
4. Select **Bar Chart**.
5. Sort the bars.

This allows us to compare:

Mumbai vs Pune vs Nashik

Step 5: Create Profit by Category

We want to find:

> **Which category generates the highest profit?**

1. Create a new worksheet.
2. Drag **Category → Columns**.
3. Drag **Profit → Rows**.
4. Select **Bar Chart**.
5. Turn on **Show Mark Labels**.

Now the profit value will appear on each bar.

Step 6: Create Sales Trend

We can analyze how sales change over time.

1. Create a new worksheet.
2. Drag **Date → Columns**.
3. Drag **Sales → Rows**.
4. Tableau will create a **line chart**.
5. If necessary, change Date to **Day** or **Month**.

This helps us answer:

> **How are sales changing over time?**

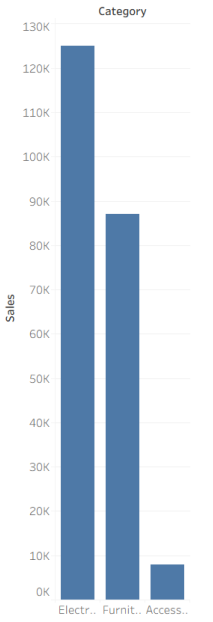
Step 7: Create a Sales by Category Pie Chart

We can also show the contribution of each category.

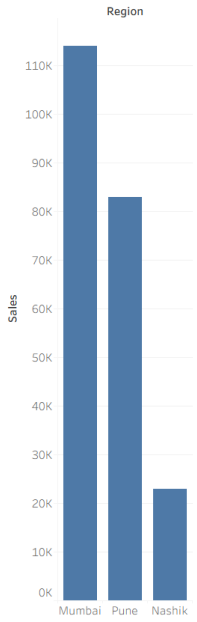
1. Create a new worksheet.
2. Drag **Category → Color**.
3. Drag **Sales → Angle**.
4. Change **Marks** from Automatic to **Pie**.
5. Drag **Sales → Label**.

This shows the proportion of total sales contributed by each category.

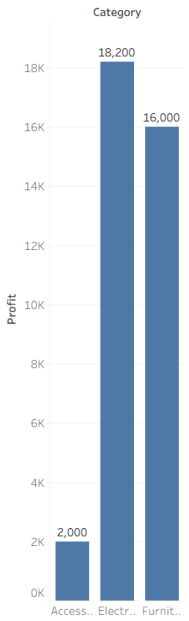
Sheet 1



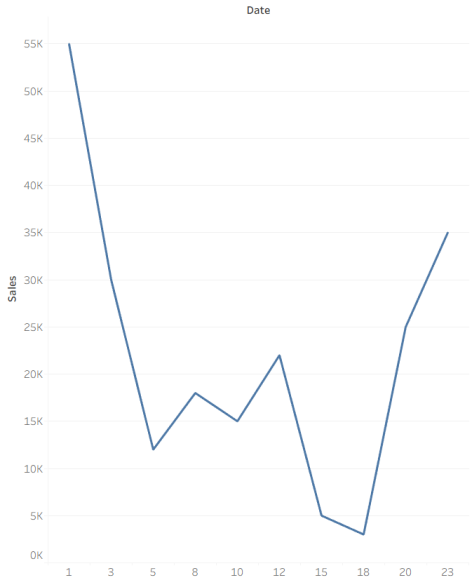
Sales by Region



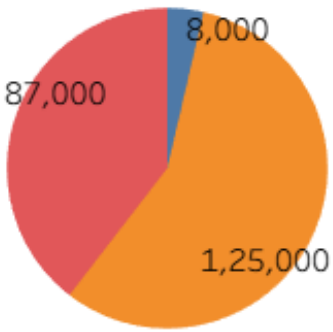
Profit by Category



Sales Trend



Sales By Category Pie Chart



Step 8: Add a Filter

This is an important part of **interactive visualization**.

For example, we can allow the user to select a **Region**.

1. Select your chart.
2. Drag **Region** to the **Filters** shelf.
3. Select **All**.
4. Click the filter drop-down.
5. Select **Show Filter**.

Now the user can select:

- Mumbai
- Pune
- Nashik

The chart will change according to the selected region.



Step 9: Create a Dashboard

Now we combine all the charts.

1. Click **New Dashboard** at the bottom.
2. Give it a name:

RETAIL STORE PERFORMANCE DASHBOARD

3. Drag the following sheets into the dashboard:
    - Sales by Category
    - Sales by Region
    - Profit by Category
    - Sales Trend
    - Sales by Category Pie Chart

Arrange them properly.

Step 10: Make the Dashboard Interactive

Add filters such as:

Region

Category

Date

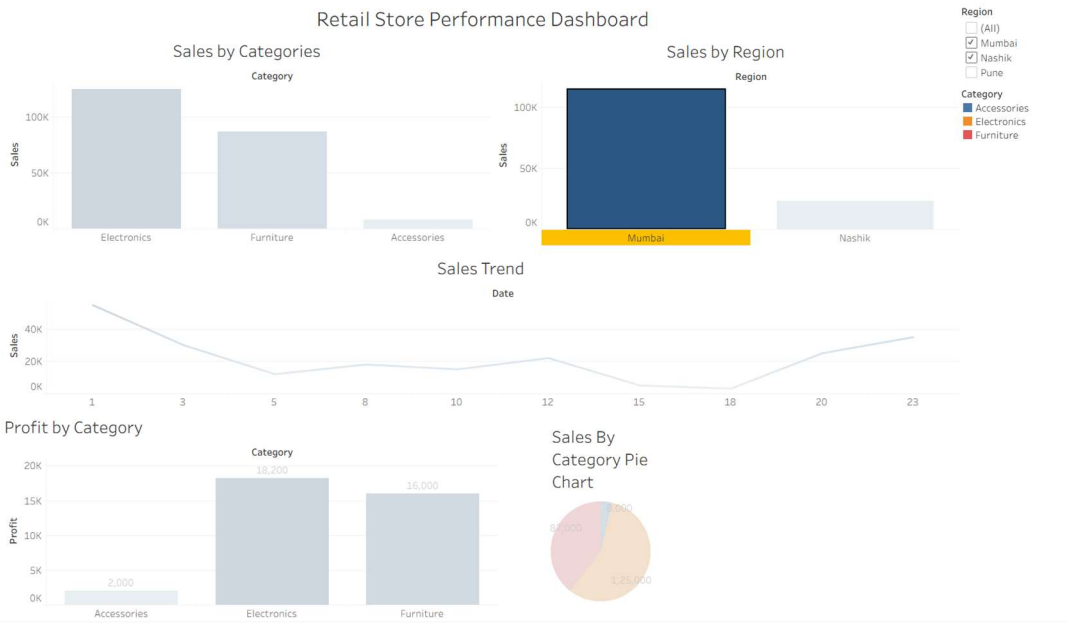
For example:

If the user selects:

Region = Mumbai

all relevant charts can update to show Mumbai's data.

This is what makes the dashboard **interactive**.



Step 11: Extract Business Insights

Finally, observe the charts and write meaningful conclusions.

For example:

Possible Insights

1. **Electronics** contributes the highest sales among the categories.
2. **Mumbai** generates higher sales compared with some other regions.
3. Furniture products contribute significant profit.
4. Sales vary across different dates.
5. The dashboard allows users to compare regions and categories interactively.

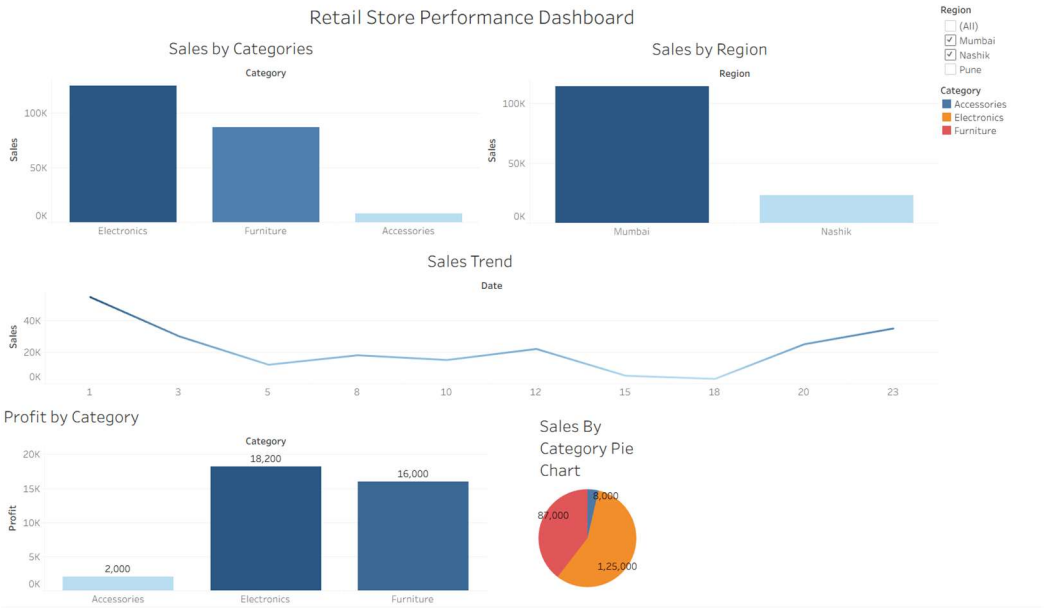
The exact insights should be written according to the results produced by your Tableau dataset.

Expected Output:-

You should finally have a Tableau dashboard containing:

RETAIL STORE PERFORMANCE DASHBOARD

- 📊 Sales by Category
- 📊 Sales by Region
- 📊 Profit by Category
- 📈 Sales Trend
- 🥧 Sales Contribution by Category
- 🔽 Region/Category filters



Practical Flow:-

Raw Dataset → Tableau → Data Analysis → Charts → Filters → Interactive Dashboard → Business Insights

Conclusion:-

Thus, this practical helped us understand how Tableau can be used as an effective Business Intelligence tool to analyze data and generate meaningful insights. We learned how to connect a dataset to Tableau, organize and analyze the data, and represent information using different charts and graphs, such as bar charts, line charts, and pie charts. We also understood how to apply filters and create interactive visualizations to explore data from different perspectives. These visualizations make complex information easier to understand and help identify trends, patterns, comparisons, and variations in business performance. Overall, this practical demonstrated how Tableau transforms raw data into useful business insights, supports data-driven decision-making, and improves the efficiency of data analysis and reporting.

PRACTICAL NO.6(B)

Aim:- Apply the concept of expected value to evaluate different decision alternatives and select the optimal one.

Code:-

```
high_demand_probability=0.60
low_demand_probability=0.40
#probabilities and payoffs
high_demand_profit=80000
low_demand_profit=20000
do_not_launch_profit=30000
ev_launch=(
 high_demand_probability*high_demand_profit
 +low_demand_probability*low_demand_profit)
ev_not_launch=do_not_launch_profit

#Display result
print("DECISION MAKING UNDER UNCERTAINITY")
print("-----------------------------------")
print("Expected value- Launch Product:", ev_launch)
print("Expected Value- Do Not Launch:", ev_not_launch)
#select the best decision
if ev_launch> ev_not_launch:
 print("Best Decision: Launch Product")
elif ev_not_launch> ev_launch:
 print("Best Decision: Do not Launch")
else:
 print("Both Decisions have the same expected value.")
```

Output:-

```
DECISION MAKING UNDER UNCERTAINITY
----------------------------------
Expected value- Launch Product: 56000.0
Expected Value- Do Not Launch: 30000
Best Decision: Launch Product
```

Concluison:-

The Decision Tree model successfully demonstrated how business decisions can be made under uncertainty by considering different possible outcomes and their associated probabilities. By calculating the **Expected Value (EV)** for each decision alternative, the outcomes were compared objectively. The alternative with the **highest expected value** was identified as the optimal decision. Thus, the practical showed that decision trees and expected value analysis help businesses reduce uncertainty, evaluate risks, and make more informed and profitable decisions.

PRACTICAL 9. Hypothesis Testing

PRACTICAL NO.9(A)

Aim:- Formulate null and alternative hypothesis related to business problem, conduct a hypothesis test during appropriate statistical test, and interpret the result.

Code:-

```
from scipy.stats import ttest_1samp

#Sample delivery times
delivery_time=[28,31,29,32,30,27,33,31,29,30]

#Claimed average
claimed_mean=30

#Perform t-test
t_stat,p_value=ttest_1samp(
    delivery_time,
    claimed_mean
)

print("T-Statistic:",t_stat)
print("P-Value:",p_value)

#Significance level
alpha=0.05

if p_value<alpha:
    print("Reject the Null Hypothesis.")
    print("Averge delivery time is significantly different from 30 minutes.")
else:
    print("Fail to reject the null hypothesis.")
    print("There is not enough evidence to say that the average delivery time is different from
30 minutes.")
```

Output:-

T-Statistic: 0.0
P-Value: 1.0
Fail to reject the null hypothesis.
There is not enough evidence to say that the average delivery time is different from 30 minutes.

PRACTICAL NO.9(B)

Aim:- Perform A/B testing on a website or marketing campaign to evaluate the effectiveness of different strategies and make data-driven decisions.

Code:-

```
from statsmodels.stats.proportion import proportions_ztest

#Purchases
purchases=[10,15]

#Visitors
visitors=[100,100]

#Perform A/B test
z_stat,p_value=proportions_ztest(
    purchases,
    visitors
)

#Calculate conversion rates
rate_A=purchases[0]/visitors[0]*100
rate_B=purchases[1]/visitors[1]*100

print("Conversion Rate A:",rate_A,"%")
print("Conversion Rate B:",rate_B,"%")

print("Z-Statistics:",z_stat)
print("P-Value:",p_value)

#Decision
alpha=0.05

if p_value<alpha:
    print("Reject the Null Hypothesis.")
    print("There is a significant difference between A and B.")
else:
    print("Fail to reject the Null Hypothesis.")
    print("There is no significant difference between A and B.")
```

Output:-

Conversion Rate A: 10.0 %
Conversion Rate B: 15.0 %
Z-Statistics: -1.0690449676496971
P-Value: 0.28504940740261286
Fail to reject the Null Hypothesis.
There is no significant difference between A and B.